Backend Engineering with Node.js: Zero to Hero

Oct 18, 2026 · @Subhamoy Ghosh

Part 0: How to use this report

This report takes you from "what is an HTTP request" to designing production backends in Node.js, in 12 parts. Every part follows the order you asked for: theory first, then libraries and alternatives, then code.

Every code step is written as a small **build card** so you can write the code yourself instead of pasting a dump:

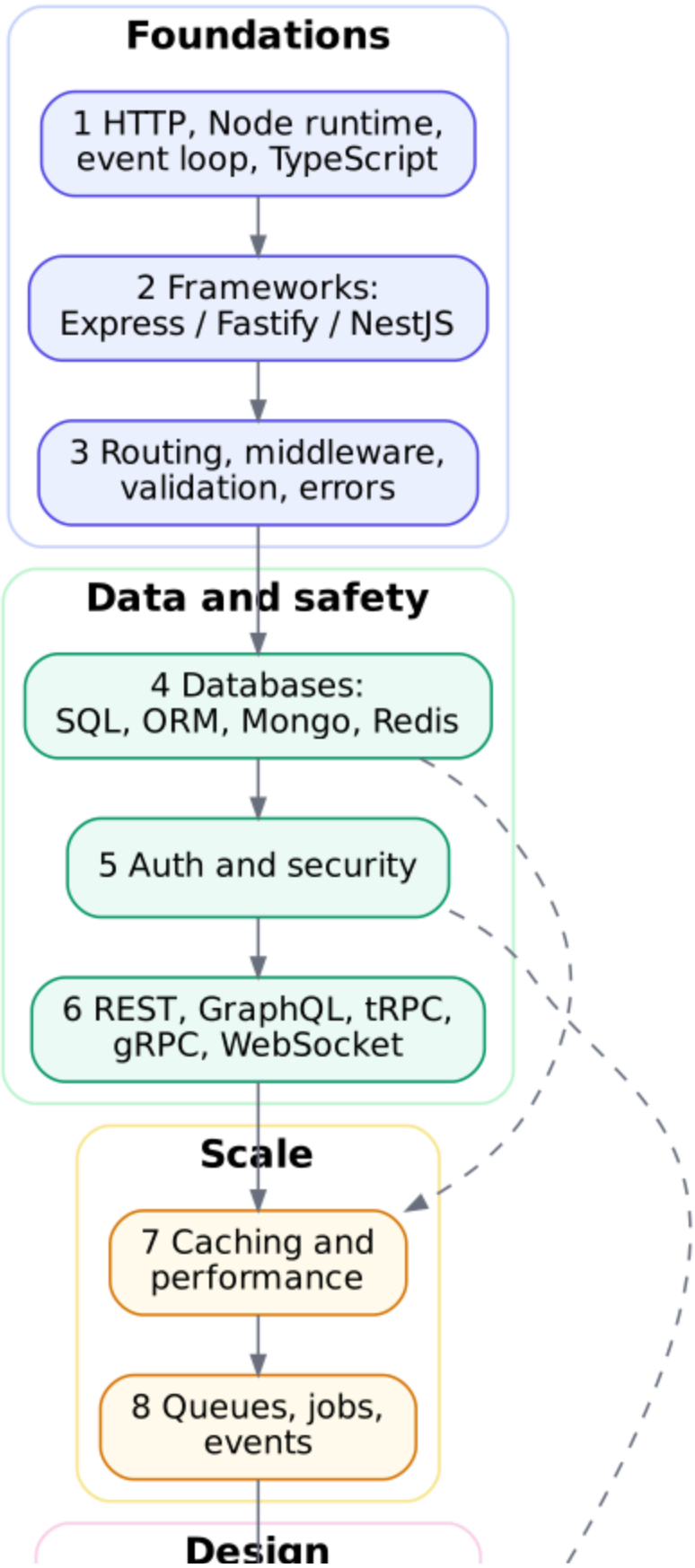
- **Aim:** what you are building in this step.
- **Input / Output:** what goes in and what must come out.
- **Check:** one command or request that proves it works.
- **Reference sketch:** a short snippet to compare against, only after you have tried it.

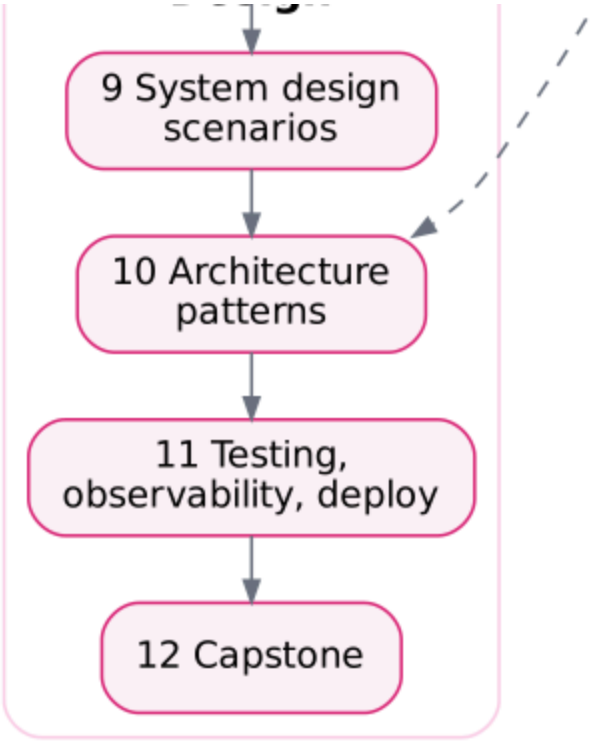
The parts build on each other, so go in order the first time.

Stage	Parts	You can do this afterwards
Foundations	1, 2, 3	Build a typed, validated REST API with any of the three frameworks
Data and safety	4, 5, 6	Persist data correctly, log users in, resist common attacks
Scale	7, 8	Cache, queue, and serve real-time traffic
Design	9, 10, 11, 12	Pick libraries and architecture for a given scenario, test, ship, and observe it

Assumptions: Node.js 22 LTS, TypeScript, PostgreSQL as the main database, Docker for local services. Examples are TypeScript-first and lean on JavaScript you already know (async/await, ES modules). Library versions move fast, so run `npm view <package> version` before installing and check current docs for exact option names.

Learning roadmap: the diagram below shows the dependency order of the topics.


Foundations
1 HTTP, Node runtime, event loop, TypeScript
2 Frameworks: Express / Fastify / NestJS
3 Routing, middleware, validation, errors
Data and safety
4 Databases: SQL, ORM, Mongo, Redis
5 Auth and security
6 REST, GraphQL, tRPC, gRPC, WebSocket
Scale
7 Caching and performance
8 Queues, jobs, events
Design



Part 1: Foundations (HTTP, Node runtime, TypeScript, tooling)

A backend is a program that waits for requests, applies business rules, reads or writes data, and sends back a response. Everything in later parts is a refinement of that loop.

1.1 Theory: HTTP in 10 minutes

A request has a **method**, a **path**, **headers**, and an optional **body**. A response has a **status code**, headers, and a body (usually JSON).

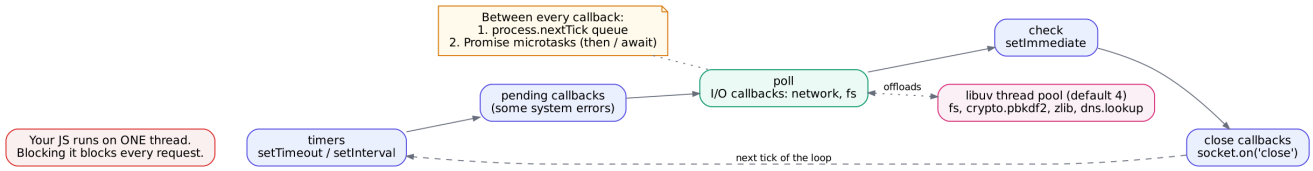
Method	Meaning	Safe (no change)	Idempotent (repeat = same result)
GET	Read a resource	Yes	Yes
POST	Create or trigger an action	No	No
PUT	Replace a resource	No	Yes
PATCH	Partially update	No	Usually not guaranteed
DELETE	Remove	No	Yes

Status range	Meaning	Common codes
2xx	Success	200 OK, 201 Created, 202 Accepted, 204 No Content
3xx	Redirect	301, 302, 304 Not Modified
4xx	Client mistake	400 bad input, 401 not logged in, 403 not allowed, 404 missing, 409 conflict, 422 validation failed, 429 rate limited
5xx	Server mistake	500, 502, 503, 504

Key headers to know now: `Content-Type`, `Authorization`, `Cookie` / `Set-Cookie`, `Cache-Control`, `ETag`, `Origin` (CORS). HTTP is **stateless**: the server does not remember you between requests unless you send something (cookie, token) that lets it look you up.

1.2 Theory: how Node runs your code

Node.js = the V8 JavaScript engine + **libuv** (event loop, networking, thread pool) + built-in modules (`http`, `fs`, `crypto`, `stream`). Your JavaScript runs on **one thread**. That is fast for I/O-heavy work (waiting on databases, networks) because while one request waits, the thread serves another. It is a disaster for CPU-heavy work (big loops, image processing, `JSON.parse` of 200 MB) because nothing else runs until it finishes.



Rules that follow from this diagram:

1. Never block the loop: avoid `*Sync` functions (`fs.readFileSync`) and heavy loops inside request handlers.
2. `process.nextTick` and Promise callbacks run before the loop moves on, so a runaway promise chain can starve I/O.
3. CPU-bound jobs go to `worker_threads`, a child process, or a queue worker (Part 8).
4. To use all CPU cores run several processes: `cluster` module, PM2, or one container per core behind a load balancer.

1.3 Theory: async, streams, and events

- **Promises and async/await** replace callbacks. `await Promise.all([a(), b()])` runs independent work in parallel; sequential `await` lines run one after another.
- **Always handle rejections.** An unhandled rejection crashes modern Node. In Express 4 an `async` handler that throws is *not* caught automatically (Express 5 does catch it).
- **Streams** process data in chunks so a 2 GB file never sits fully in memory: `pipeline(readable, transform, writable)` from `node:stream/promises`.
- **EventEmitter** (`node:events`) is the base of most Node objects: `emitter.on('event', fn)`, `emitter.emit('event', data)`. You will use the idea for domain events in Part 8.
- **AbortController** cancels fetches and timers; use it for request timeouts.

1.4 Libraries and tools for this stage

Need	Pick	Alternatives	Notes
Runtime	Node.js 22 LTS	Bun, Deno	Bun and Deno are faster to start and ship TypeScript, but Node has the widest library and hosting support
Package manager	pnpm	npm, yarn	pnpm is fast and strict about dependencies
TypeScript runner	tsx	ts-node, `node --experimental-strip-types`, SWC	Recent Node 22 releases can strip types natively; tsx is the safe choice everywhere
Compiler / bundler	tsc, tsup	SWC, esbuild	tsc for type checking, tsup for fast builds
Auto restart	`node --watch`	nodemon, tsx watch	
Env variables	`node --env-file=.env`	dotenv	Never commit `.env`
Lint / format	ESLint + Prettier	Biome	Biome is one fast tool for both

Need	Pick	Alternatives	Notes
Dates	date-fns	Luxon, Day.js	Store UTC, convert at the edge

1.5 Build cards

Card 1.A: a bare HTTP server (no framework)

- **Aim:** feel what frameworks do for you.
- **Input:** `GET /health`, `GET /time`, anything else.
- **Output:** `200 {"status":"ok"}`, `200 {"now":"<ISO date>"}`, `404 {"error":"not found"}`, all as JSON with the right `Content-Type`.
- **Check:** `curl -i localhost:3000/health`, then an unknown path.
- **Reference sketch (try yourself first):**

```
import http from 'node:http';

const server = http.createServer((req, res) => {
  const send = (code: number, body: unknown) => {
    res.writeHead(code, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify(body));
  };
  if (req.method === 'GET' && req.url === '/health') return send(200, {
status: 'ok' });
  if (req.method === 'GET' && req.url === '/time') return send(200, { now:
new Date().toISOString() });
  send(404, { error: 'not found' });
});
server.listen(3000);
```

Card 1.B: prove the event loop blocks

- **Aim:** see why one slow handler hurts everyone.
- **Input:** add `GET /block` that runs `while (Date.now() - start < 5000) {}`.
- **Output:** while `/block` runs, `/health` hangs too.
- **Check:** open two terminals; run `curl localhost:3000/block` and immediately `curl localhost:3000/health`. Then move the loop into a `worker_threads` Worker and repeat: `/health` must answer instantly.

Card 1.C: a TypeScript project skeleton

- **Aim:** a repeatable starting point for every project below.
- **Input:** empty folder.
- **Output:** `package.json` with `"type": "module"`, `tsconfig.json` (strict mode), `src/index.ts`, scripts `dev`, `build`, `start`, `typecheck`.
- **Check:** `pnpm dev` restarts when you edit; `pnpm typecheck` fails when you assign a string to a number.

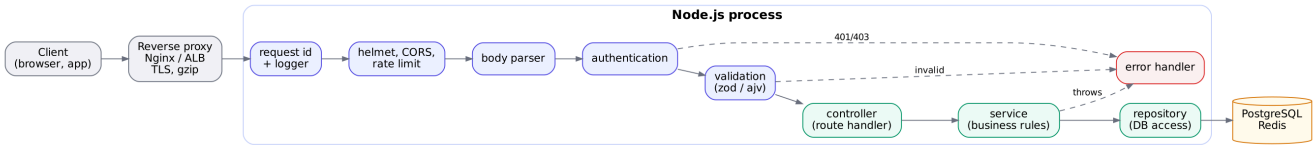
Self-check questions: Why can one Node process serve thousands of waiting requests but not two heavy loops? What is the difference between 401 and 403? Why is PUT idempotent and POST not?

Part 2: Frameworks (Express, Fastify, NestJS and friends)

A web framework turns the raw `http` server from Card 1.A into something you can organise: routing, middleware, body parsing, error handling, and plugins. They differ mostly in **how much structure they force on you**.

2.1 Theory: the request lifecycle

Every framework passes a request through a chain of steps before your handler runs, and the response back through error handling. Learn this chain once; all frameworks are variations of it.



- **Middleware** is a function that can read or change the request, end the response, or call `next()` to continue the chain. Order matters: logging first, auth before routes, error handler last.
- **Controller / route handler** translates HTTP into a function call and back. It should contain almost no business logic.
- **Service** holds business rules. **Repository** talks to the database. This split is what makes code testable (Part 10).

2.2 The frameworks compared

Framework	Style	Strengths	Weaknesses	Choose it when
Express 5	Minimal, middleware-based	Largest ecosystem, every tutorial uses it, easy to learn	No built-in validation, structure, or TypeScript help; slower than Fastify	Learning, small APIs, quick prototypes, legacy code
Fastify 5	Minimal but fast, plugin-based	JSON-schema validation, fast serialisation, built-in pino logging, strong TypeScript support, encapsulated plugins	Smaller ecosystem than Express; plugin model needs a learning step	Performance-sensitive APIs, microservices, teams that want schema-first design
NestJS 11	Opinionated, Angular-like (modules, decorators, dependency injection)	Enforced structure, DI makes testing easy, first-class support for GraphQL, WebSockets, queues, microservices, OpenAPI; runs on Express or Fastify underneath	Steeper learning curve, decorator magic, more boilerplate	Large teams, long-lived enterprise apps, many modules
Hono	Tiny, web-standard Request/Response	Runs on Node, Bun, Deno, Cloudflare Workers; very small	Younger ecosystem	Edge/serverless APIs, portable code

Framework	Style	Strengths	Weaknesses	Choose it when
Koa	Minimal, async middleware by the Express authors	Clean async model	Small ecosystem, little momentum now	Rare; mostly legacy
AdonisJS	Full-stack, Laravel-like	Batteries included: ORM, auth, validation	Smaller community	You want Rails/Laravel style in Node
tRPC, Elysia (Bun)	Type-safe RPC / Bun-first	End-to-end types	Narrower use cases	TypeScript monorepos (Part 6)

Practical advice: learn Express first because you will meet it everywhere, then build the same API in Fastify and NestJS. After that you will see that the concepts (routing, middleware, validation, DI) are the same and only the spelling changes.

2.3 Express in practice

```
import express from 'express';
const app = express();
app.use(express.json());

app.get('/notes/:id', (req, res) => {
  res.json({ id: req.params.id });
});

// error handler: four arguments, registered last
app.use((err: Error, _req: express.Request, res: express.Response, _next:
express.NextFunction) => {
  res.status(500).json({ error: 'internal error' });
});
app.listen(3000);
```

Common Express companions: `helmet` (security headers), `cors`, `morgan` or `pino-http` (request logs), `compression`, `express-rate-limit`, `multer` (file uploads), `cookie-parser`.

2.4 Fastify in practice

```
import Fastify from 'fastify';
const app = Fastify({ logger: true });

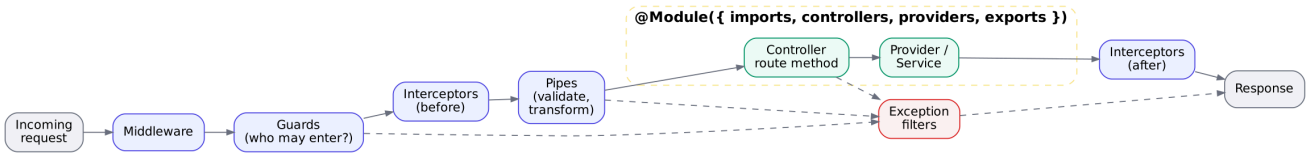
app.get('/notes/:id', {
  schema: {
    params: { type: 'object', properties: { id: { type: 'string' } },
required: ['id'] },
    response: { 200: { type: 'object', properties: { id: { type: 'string' } }
} },
  },
}, async (req) => ({ id: (req.params as { id: string }).id }));

await app.listen({ port: 3000 });
```

Fastify ideas to learn: **plugins** (`app.register(plugin, { prefix: '/v1' })`) give encapsulation; **hooks** (`onRequest` , `preHandler` , `onSend`) replace middleware; **decorators** attach shared objects such as a database client. Companions: `@fastify/cors` , `@fastify/helmet` , `@fastify/rate-limit` , `@fastify/jwt` , `@fastify/multipart` , `@fastify/swagger` , and type providers (`@fastify/type-provider-typebox` or `fastify-type-provider-zod`) for end-to-end types.

2.5 NestJS in practice

Nest organises code into **modules**. Each module bundles controllers (HTTP), providers (services, repositories), and what it imports or exports. The framework builds objects for you and injects them (**dependency injection**), so a controller just declares what it needs.



```
@Injectable()
export class NotesService {
  findOne(id: string) { return { id }; }
}

@Controller('notes')
export class NotesController {
  constructor(private readonly notes: NotesService) {}

  @Get(':id')
  findOne(@Param('id') id: string) { return this.notes.findOne(id); }
}

@Module({ controllers: [NotesController], providers: [NotesService] })
export class NotesModule {}
```

Scaffold with `npx @nestjs/cli new project`. Know these five building blocks: **Module, Controller, Provider, Pipe** (validate/transform input), **Guard** (allow or deny), plus **Interceptor** (wrap handlers: logging, caching, response mapping) and **Exception filter** (shape errors). Switch the HTTP engine to Fastify with `@nestjs/platform-fastify` when you want its speed.

2.6 Build cards

Card 2.A: Notes API in Express

- **Aim:** a CRUD API with an in-memory array, so the framework is the only new thing.
- **Input:** `POST /notes {title, body}`, `GET /notes`, `GET /notes/:id`, `PATCH /notes/:id`, `DELETE /notes/:id`.
- **Output:** `201` with the created note and an `id`; `200` lists; `404` for unknown id; `204` on delete.
- **Check:** write 6 `curl` commands (or a `.http` file for the REST Client extension) that exercise every status code above.

Card 2.B: the same API in Fastify

- **Aim:** compare how schemas replace manual checks.
- **Input / Output:** identical to 2.A, but each route has a JSON schema for body and response.

- **Check:** sending `{ "title": 5 }` returns `400` automatically without you writing an `if`. Note the difference in code size and speed with `npx autocannon -c 50 -d 10 localhost:3000/notes`.

Card 2.C: the same API in NestJS

- **Aim:** learn modules and DI.
- **Input / Output:** identical, split into `NotesModule`, `NotesController`, `NotesService`.
- **Check:** the service has no HTTP types in it; you can call `new NotesService()` in a plain script and it works.

Self-check questions: What does `next()` do in Express? Why does Fastify need schemas for fast responses? What problem does dependency injection solve in a 50-file project?

Part 3: API building blocks (routing, validation, errors, config, logging)

These five concerns appear in every backend. Getting them right early prevents most production pain.

3.1 Theory

Routing and resource design. Model nouns, not verbs: `/users`, `/users/42`, `/users/42/orders`. Use plural names, nest only one level, and put filters in the query string (`/orders?status=paid&limit=20`). Version in the path (`/v1/...`) or a header. Return the same JSON shape every time.

Validation. Never trust input. Validate **at the edge** (body, params, query, headers) and convert it into typed objects before it reaches your services. Reject with `400` or `422` and a machine-readable message. Validate output too if the shape matters to clients.

Error handling. Separate *expected* errors (not found, conflict, invalid input) from *bugs*. Throw typed errors in services (`NotFoundError`, `ConflictError`), and map them to HTTP codes in **one** central error handler. Never leak stack traces to clients. A consistent error body helps every client:

```
{ "error": { "code": "NOTE_NOT_FOUND", "message": "Note 42 does not exist",
"requestId": "b7f3..." } }
```

Configuration. Follow the twelve-factor rule: config lives in environment variables, never in code. Parse and validate them **once at startup** so the app refuses to boot when something is missing.

Logging. Log structured JSON (one object per line) with a request id, not free text. Levels: `error`, `warn`, `info`, `debug`. Never log passwords, tokens, or full card numbers. Redact them in the logger configuration.

3.2 Libraries and alternatives

Concern	Pick	Alternatives	Trade-off
Validation (schema in TypeScript)	Zod	Valibot, ArkType, Yup, Joi, TypeBox, Ajv	Zod has the best ergonomics and types; Valibot is smaller; TypeBox/Ajv are fastest and match Fastify JSON schema
Validation (class-based)	class-validator + class-transformer	Zod pipes	Standard in NestJS; decorators on DTO classes
Env config	Zod parse of `process.env`	envalid, `@nestjs/config`, convict	Fail fast with clear messages
Logging	pino	winston, bunyan, console	pino is the fastest and Fastify's default; pipe through `pino-pretty` in development
HTTP request logs	pino-http	morgan	morgan is simple text, pino-http is structured
Request id	`crypto.randomUUID()` + `AsyncLocalStorage`	cls-hooked, `nestjs-cls`	AsyncLocalStorage carries the id through async calls without passing it by hand

Concern	Pick	Alternatives	Trade-off
Error shape	Own `AppError` classes	`http-errors`, `@hapi/boom`	Small and enough; RFC 9457 "problem details" is the standard format if you want one
Docs	OpenAPI via `@fastify/swagger`, `@nestjs/swagger`, `zod-to-openapi`	Swagger JSDoc	Generate the spec from the same schemas you validate with
IDs	UUID v7 / ULID	auto-increment, nanoid, cuid2	Time-ordered ids index better than random UUID v4

3.3 Build cards

Card 3.A: validation middleware (Express + Zod)

- **Aim:** one reusable `validate(schema)` that checks `body`, `params`, `query`.
- **Input:** a Zod schema `{ title: string (1..100), body: string optional }`.
- **Output:** on success the handler receives typed data; on failure `400` with a list of `{ path, message }`.
- **Check:** post `{}` and `{"title":""}` and `{"title":"ok"}`: first two fail with readable messages, third passes.
- **Reference sketch:**

```
import { z, ZodTypeAny } from 'zod';
export const validate = (schema: ZodTypeAny) => (req, res, next) => {
  const r = schema.safeParse(req.body);
  if (!r.success) return res.status(400).json({ error: { code: 'VALIDATION',
issues: r.error.issues } });
  req.body = r.data;
  next();
};
```

Card 3.B: central error handler

- **Aim:** services throw typed errors, one handler turns them into responses.
- **Input:** classes `AppError(status, code, message)`, `NotFoundError`, `ConflictError`.

- **Output:** known errors give their status and code; unknown errors give `500` with a generic message and are logged with the stack.
- **Check:** make `GET /notes/999` throw `NotFoundError` (expect 404), and make another route throw a plain `Error` (expect 500, no stack in the body).

Card 3.C: typed config

- **Aim:** crash at startup when config is wrong.
- **Input:** `PORT`, `DATABASE_URL`, `JWT_SECRET`, `NODE_ENV`.
- **Output:** an exported frozen `config` object with correct types (`PORT` is a number).
- **Check:** start the app without `DATABASE_URL`: it must exit with a message naming the missing variable.

Card 3.D: structured logging with request ids

- **Aim:** trace one request through all its log lines.
- **Input:** a middleware that creates `requestId` (or reads `x-request-id`), stores it in `AsyncLocalStorage`, and sets it on the response header.
- **Output:** every log line for that request contains the same `requestId`, and the error body from 3.B includes it.
- **Check:** call an endpoint that logs in two different functions; grep the output for one id and see both lines.

Self-check questions: Why validate at the edge instead of inside the service? Why return `409` for a duplicate email and not `400`? What breaks if you read `process.env.X` all over the code base?

Part 4: Databases (SQL, ORMs, MongoDB, Redis)

Most backend bugs and slowdowns live in the data layer, so this part is long on purpose.

4.1 Theory: relational databases (PostgreSQL)

- **Tables, rows, keys.** A primary key identifies a row; a foreign key links rows (`orders.user_id` references `users.id`). Relationships: one-to-many, many-to-many (via a join table).
- **Normalisation.** Store each fact once. Duplicate data only on purpose, for speed, and know why.

- **Indexes.** A B-tree index turns a full table scan into a lookup. Index columns you filter, join, or sort on. Every index slows writes and uses disk, so add them based on slow queries, checked with `EXPLAIN ANALYZE`. A composite index `(user_id, created_at)` serves queries that filter by `user_id` and sort by `created_at`.
- **Transactions (ACID).** A group of statements that all succeed or all roll back. Needed whenever two writes must stay consistent (debit one account, credit another). Isolation levels (read committed, repeatable read, serializable) trade safety for concurrency; PostgreSQL defaults to read committed.
- **Constraints.** `NOT NULL`, `UNIQUE`, `CHECK`, foreign keys put your rules in the database, where no bug in the app can bypass them.
- **Connection pooling.** Opening a connection is expensive and PostgreSQL has a hard connection limit. Keep a pool (typically 10 to 20 per app instance) and use PgBouncer or a managed pooler when you have many instances.
- **The N+1 problem.** Loading 100 posts and then querying the author 100 times. Fix with a join, an `IN` batch, or a loader (DataLoader in GraphQL).
- **Pagination.** `OFFSET` gets slower as pages go deeper. **Keyset (cursor) pagination** (`WHERE id < :last ORDER BY id DESC LIMIT 20`) stays fast.
- **Migrations.** Schema changes are versioned files applied in order. Never edit the production schema by hand. For zero downtime: add column nullable, deploy code that writes both, backfill, then enforce and remove the old column.

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1 AND balance >= 100;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;   -- or ROLLBACK on any error

CREATE INDEX CONCURRENTLY idx_orders_user_created ON orders (user_id,
created_at DESC);
```

4.2 Theory: NoSQL and when to choose it

Type	Example	Good for	Weak at
Relational	PostgreSQL, MySQL	Most business data, reporting, strong consistency, joins	Extremely wide horizontal write scaling needs extra work

Type	Example	Good for	Weak at
Document	MongoDB	Flexible, nested data that is read as one unit (CMS content, catalogues)	Multi-document relations and transactions are clumsier
Key-value	Redis, DynamoDB	Caching, sessions, counters, leaderboards, rate limits	Ad hoc queries
Search	Elasticsearch, OpenSearch, Meilisearch, Typesense	Full-text search, faceting, autocomplete	Source of truth (keep it a secondary copy)
Wide column / time series	Cassandra, ClickHouse, TimescaleDB	Huge write volume, analytics, metrics	Transactions, joins
Graph	Neo4j	Deep relationship traversal	General CRUD

Default advice: start with PostgreSQL. It also handles JSON (`jsonb`), full-text search, queues (`SKIP LOCKED`), and geospatial (PostGIS), so you can postpone extra systems until a measured need appears.

4.3 Libraries and alternatives

Level	Pick	Alternatives	Trade-off
Raw driver	`pg` (node-postgres)	postgres.js, mysql2, better-sqlite3	Full control, you write SQL and map rows
Query builder	Kysely	Knex	Type-safe SQL, close to the metal; Knex is older, less type-safe

Level	Pick	Alternatives	Trade-off
ORM (schema-first, generated client)	Prisma		Great developer experience and types; own schema language; historically heavier runtime; check current docs for the query engine details
ORM (SQL-like, TypeScript schema)	Drizzle		Thin, close to SQL, lightweight, fast-growing
ORM (decorator, Active Record / Data Mapper)	TypeORM, MikroORM	Sequelize	TypeORM is common in Nest; MikroORM has a cleaner unit-of-work model; Sequelize is legacy-heavy
MongoDB	Mongoose	Official `mongodb` driver, Typegoose	Mongoose adds schemas and validation
Redis	ioredis	`redis` (node-redis)	Both are fine; ioredis is popular with BullMQ
Migrations	Prisma Migrate / drizzle-kit	node-pg-migrate, Knex migrations, Flyway, Atlas	Use the tool that matches your ORM
Testing DB	Testcontainers	docker-compose, pg-mem	Real PostgreSQL in a throwaway container

Choosing: want fastest start and great types, use Prisma. Want SQL control with types, use Drizzle or Kysely. Writing complex reports, drop to raw SQL for those queries; no ORM does everything well.

```
// Prisma: transaction + include (avoids N+1)
const order = await prisma.$transaction(async (tx) => {
  const o = await tx.order.create({ data: { userId, items: { create: items }
} });
  await tx.stock.updateMany({ where: { sku: { in: skus } }, data: { qty: {
decrement: 1 } } });
  return o;
});
const posts = await prisma.post.findMany({ take: 20, include: { author: true
} });
```

4.4 Redis in 5 minutes

Redis is an in-memory data structure server. Core types: **string** (cache values, counters with `INCR`), **hash** (an object), **list** (simple queue), **set**, **sorted set** (leaderboards, delayed jobs), **stream** (event log), plus **pub/sub**. Every key can expire (`SET key value EX 60`). Data is lost on restart unless persistence (RDB/AOF) is configured, so treat it as a cache or a fast helper store unless you deliberately design otherwise.

4.5 Build cards

Card 4.A: PostgreSQL in Docker plus a repository

- **Aim:** replace the in-memory array of Card 2.A with a real table.
- **Input:** a `docker-compose.yml` with `postgres:16` (or newer), a migration creating `notes(id, title, body, created_at)`.
- **Output:** a `NotesRepository` with `create`, `findById`, `list`, `update`, `delete`; the HTTP layer only calls the repository.
- **Check:** restart the server and your notes are still there. `psql` shows the rows.

Card 4.B: relations and N+1

- **Aim:** see N+1 and fix it.
- **Input:** tables `users` and `posts` (one user has many posts); an endpoint `GET /posts` returning each post with its author name.
- **Output:** first version runs 1 + N queries (log them); second version runs 1 or 2.
- **Check:** turn on query logging and count statements for 50 posts before and after.

Card 4.C: transaction with a rollback

- **Aim:** all-or-nothing writes.

- **Input:** `POST /transfer {from, to, amount}` using two accounts.
- **Output:** insufficient balance gives `409` and **no** row changes; success changes both rows.
- **Check:** fire 20 parallel transfers with a script and confirm the total balance never changes.

Card 4.D: index and keyset pagination

- **Aim:** measure what an index does.
- **Input:** seed 1 million rows (use `generate_series`); query one user's latest 20 rows.
- **Output:** `EXPLAIN ANALYZE` before (sequential scan) and after (index scan) with the timing difference; an endpoint using a `cursor` query parameter.
- **Check:** record both timings in your notes.

Self-check questions: What does a transaction guarantee that two separate queries do not? Why can an index make writes slower? When would you pick MongoDB over PostgreSQL, honestly?

Part 5: Authentication, authorization, and security

Authentication answers "who are you?". Authorization answers "what may you do?". Confusing the two is the root of many breaches.

5.1 Theory: passwords

Store only a **slow, salted hash**, never the password or a fast hash such as SHA-256. Use **argon2id** (preferred) or bcrypt. Enforce a minimum length, check against breached-password lists if you can, and rate-limit login attempts. Return the same error for "unknown email" and "wrong password" so attackers cannot enumerate accounts.

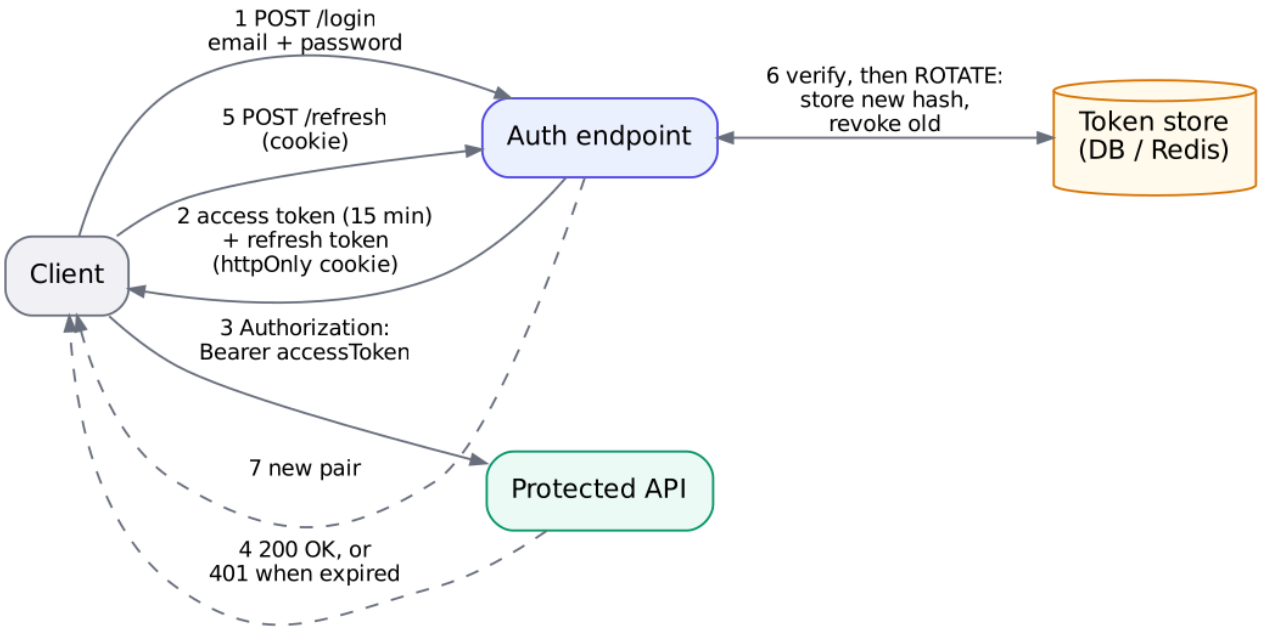
5.2 Theory: sessions versus tokens

	Server session	JWT (stateless token)
What the client holds	Random session id in a cookie	Signed token containing claims (user id, role, expiry)
Where state lives	Server store (Redis or DB)	In the token itself
Revoke instantly	Yes, delete the session	Hard; need a deny list or short expiry

	Server session	JWT (stateless token)
Scales across servers	Needs shared store	Any server can verify with the key
Best for	Browser apps with your own backend	Mobile apps, service-to-service, third-party APIs

Honest advice: for a normal web app with a single backend, server sessions in an `httpOnly`, `Secure`, `SameSite` cookie are simpler and safer than JWT. Use JWT when you really need stateless verification across services.

The common JWT design is a **short-lived access token** (about 15 minutes) plus a **long-lived refresh token** that is stored hashed in the database, sent in an `httpOnly` cookie, and **rotated** on each use (the old one is revoked; reuse of a revoked token means theft and kills the whole family).



JWT rules: verify the signature **and** `exp`, `iss`, `aud`; pin the algorithm (never accept `none`); keep secrets or private keys out of code; do not put sensitive data in the payload (it is only encoded, not encrypted).

5.3 Theory: OAuth 2.0 and OpenID Connect

Use OAuth 2.0 / OIDC for "Sign in with Google/GitHub" and for protecting APIs with an identity provider. Today's browser and mobile flow is the **Authorization Code flow with PKCE**. OAuth gives *access* to resources; **OIDC** adds an *identity* layer (the ID token). Do not invent your own protocol; use a vetted library or provider.

5.4 Theory: authorization

- **RBAC** (role-based): `admin`, `editor`, `viewer`. Simple, covers most apps.
- **ABAC / policy-based**: rules on attributes ("editors may edit documents in their own team").
- **Ownership checks**: always check that the resource belongs to the caller (`WHERE id = :id AND owner_id = :userId`). Missing this is the "broken access control" bug, number one on the OWASP Top 10.
- Enforce on the **server**. Hiding a button in the UI is not security.

5.5 Theory: the attacks you must know

Attack	Defence
SQL / NoSQL injection	Parameterised queries or an ORM; never concatenate input into queries; validate types
XSS	Escape output, set a Content-Security-Policy (helmet), `httpOnly` cookies
CSRF	`SameSite` cookies, CSRF tokens for cookie-authenticated forms
Brute force / credential stuffing	Rate limit by IP and account, lockout or backoff, MFA
SSRF (server fetches attacker URL)	Allow-list hosts, block private IP ranges, no redirects
Mass assignment	Whitelist fields in your DTO/schema, never spread `req.body` into a DB update
Insecure dependencies	`npm audit`, Dependabot or Renovate, lock files, minimal dependencies

Attack	Defence
Secrets leaks	Environment variables or a secrets manager, never commit them, rotate on leak
Missing TLS	HTTPS everywhere, HSTS header
Path traversal / unsafe upload	Never use user filenames as paths, check MIME and size, store outside the web root

5.6 Libraries and alternatives

Need	Pick	Alternatives
Password hashing	`argon2`	`bcrypt`, `bcryptjs` (pure JS, slower), Node `crypto.scrypt`
JWT	`jose`	`jsonwebtoken`, `@fastify/jwt`, `@nestjs/jwt`
Sessions	`express-session` + `connect-redis`	`@fastify/session`, `iron-session`, `cookie-session`
Strategy framework	Passport.js (`passport-local`, `passport-jwt`, `passport-google-oauth20`)	`@nestjs/passport` wraps it; Arctic + Oslo for OAuth helpers
Auth library (full)	Better Auth	Auth.js (NextAuth), Lucia-style DIY with sessions
Hosted identity	Auth0, Clerk, Supabase Auth, Firebase Auth, AWS Cognito	Self-host Keycloak or Ory
Authorization	CASL, Casbin	Hand-rolled guards
Security headers	`helmet` / `@fastify/helmet`	Set manually
CORS	`cors` / `@fastify/cors`	Nest `enableCors`
Rate limiting	`express-rate-limit`, `@fastify/rate-limit`, `rate-limiter-flexible`	Nginx, API gateway, Cloudflare

Need	Pick	Alternatives
MFA	`otplib` (TOTP), WebAuthn via `@simplewebauthn/server`	Provider MFA

Build or buy? Authentication is hard to get fully right. For a product, strongly consider a hosted provider or a maintained library. Build it yourself once for learning, then choose deliberately.

5.7 Build cards

Card 5.A: register and login with argon2

- **Aim:** safe password storage.
- **Input:** `POST /auth/register {email, password}`, `POST /auth/login {email, password}`.
- **Output:** register returns `201` without the hash; login returns tokens or sets a cookie; wrong credentials return an identical `401`.
- **Check:** look in the database: the `password_hash` column starts with `$argon2id$`; two users with the same password have different hashes.

Card 5.B: access token plus rotating refresh token

- **Aim:** implement the diagram above.
- **Input:** login result, `POST /auth/refresh`, `POST /auth/logout`.
- **Output:** access token expires in 15 minutes; refresh stored as a hash with `user_id`, `family_id`, `expires_at`, `revoked_at`; reusing an old refresh token revokes the family.
- **Check:** call refresh twice with the same token: the second call fails and the newest token also stops working.

Card 5.C: auth guard and role check

- **Aim:** protect routes in one place.
- **Input:** middleware `requireAuth` (verifies token, sets `req.user`) and `requireRole('admin')`.
- **Output:** `401` without token, `403` with the wrong role, `200` otherwise.
- **Check:** a table of 3 users by 3 routes with the expected status, run as a script.

Card 5.D: hardening pass

- **Aim:** apply the attack table.
- **Input:** add helmet, CORS with an explicit origin list, rate limit of 5 login attempts per minute per IP, body size limit.
- **Output:** the 6th login attempt returns `429`; response headers include `Strict-Transport-Security` and `X-Content-Type-Options`.
- **Check:** `curl -i` and a quick loop; then run `npm audit`.

Self-check questions: Why is SHA-256 wrong for passwords? Why rotate refresh tokens? Write the SQL that prevents user A from reading user B's order.

Part 6: API styles and real-time communication

How clients talk to your server is a design decision with real consequences. This part compares the options and then covers real-time transports.

6.1 Theory: choosing an API style

Style	Idea	Strengths	Weaknesses	Use when
REST over JSON	Resources and HTTP verbs	Universal, cacheable, easy to debug, tooling everywhere	Over- and under-fetching, many round trips for complex screens	Public APIs, most CRUD apps
GraphQL	One endpoint, client asks for exact fields	No over-fetching, one request per screen, typed schema	Caching and rate limiting are harder, N+1 risk, query-cost attacks	Many clients with different data needs, complex nested data
tRPC	Call server functions from a TypeScript client with inferred types	Zero schema duplication, very fast iteration	TypeScript only, tight coupling, not for public APIs	Full-stack TypeScript monorepo (for example Next.js + Node)

Style	Idea	Strengths	Weaknesses	Use when
gRPC	Binary Protobuf contracts over HTTP/2	Fast, strict contracts, streaming, code generation in many languages	Not browser-friendly without a proxy, harder to debug	Service-to-service calls, polyglot microservices
WebSocket	Persistent two-way connection	Low latency both ways	Stateful connections complicate scaling	Chat, games, live collaboration
Server-Sent Events (SSE)	One-way stream from server over HTTP	Simple, auto-reconnect, works through proxies	Server to client only	Notifications, live feeds, streaming AI output
Webhooks	Server calls the client's URL on events	Decoupled integration	Needs retries, signatures, idempotency	Payments, third-party integrations

6.2 Theory: designing good REST

- **Status codes and shapes are part of the contract.** Pick one envelope and keep it.
- **Idempotency keys.** For `POST` that must not run twice (payments), accept an `Idempotency-Key` header, store the key with the result, and return the stored result on a retry.
- **Pagination.** Offset (`?page=3&limit=20`) is simple; cursor (`?after=abc&limit=20`) is stable and fast. Return a `nextCursor` .
- **Filtering and sorting.** `?status=paid&sort=-createdAt` , with an allow-list of sortable fields.
- **Versioning.** `/v1` in the path is the most common. Add fields freely; remove or rename only in a new version.
- **Caching.** `ETag` + `If-None-Match` returns `304` and saves bandwidth. `Cache-Control` for public data.
- **Rate-limit headers.** `RateLimit-*` or `Retry-After` on `429` .
- **Documentation.** Publish an OpenAPI (Swagger) document generated from your schemas. Clients, mocks, and tests can be generated from it.

6.3 Theory: real-time

WebSocket upgrades an HTTP connection into a persistent two-way channel. Each connection holds memory on one server, so with several servers you need a way to broadcast across them (Redis pub/sub, covered in Part 9). **Socket.IO** adds rooms, acknowledgements, automatic reconnection, and fallbacks on top of WebSocket. **SSE** is often enough: if only the server pushes, choose SSE for its simplicity.

Heartbeats (ping/pong) detect dead connections; authenticate during the handshake; limit message size and rate per connection.

6.4 Libraries and alternatives

Need	Pick	Alternatives
OpenAPI from code	`@fastify/swagger`, `@nestjs/swagger`	`zod-to-openapi`, `tsoa`, `ts-rest`, hand-written spec with Redocly
GraphQL server	GraphQL Yoga	Apollo Server, Mercurius (Fastify), `@nestjs/graphql`
GraphQL schema building	Pothos (code-first)	TypeGraphQL, Nexus, schema-first SDL
Batching to fix N+1	`dataloader`	Prisma/Drizzle joins
tRPC	`@trpc/server` + `@trpc/client`	`ts-rest`, `oRPC`, Hono RPC
gRPC	`@grpc/grpc-js` + `@grpc/proto-loader`	`nice-grpc`, Connect (`@connectrpc/connect`), `@nestjs/microservices`
WebSocket	`ws`	`uWebSockets.js` (very fast), `@fastify/websocket`, `@nestjs/websockets`
Rooms, reconnects	Socket.IO + `@socket.io/redis-adapter`	Ably, Pusher, Centrifugo (hosted or standalone)
SSE	Plain `text/event-stream` response	`better-sse`

Need	Pick	Alternatives
Webhook verification	HMAC signature check with `crypto.timingSafeEqual`	Provider SDK helpers

6.5 Build cards

Card 6.A: production-style REST resource

- **Aim:** apply the REST checklist to the Notes API.
- **Input:** `GET /v1/notes?limit=20&after=<cursor>&sort=-createdAt`, `ETag` support, OpenAPI document served at `/docs`.
- **Output:** list responses contain `data` and `nextCursor`; a repeated request with `If-None-Match` returns `304`.
- **Check:** `curl -i` twice and compare; open `/docs` and call an endpoint from the UI.

Card 6.B: idempotent POST

- **Aim:** make a retry safe.
- **Input:** `POST /v1/payments` with header `Idempotency-Key: abc`.
- **Output:** first call creates the payment (`201`); second call with the same key returns the same body and does not create another row.
- **Check:** send the same request 10 times in parallel; the table has one row (use a unique index on the key to settle races).

Card 6.C: GraphQL version of Notes

- **Aim:** feel the difference from REST.
- **Input:** a schema with `Note`, `User`, query `notes(first, after)`, mutation `createNote`.
- **Output:** one query fetches notes and their authors; a DataLoader makes it a constant number of DB queries.
- **Check:** log SQL statements while fetching 50 notes with authors.

Card 6.D: live notifications with SSE

- **Aim:** server push without WebSocket.
- **Input:** `GET /events` with `Content-Type: text/event-stream`; a `POST /notes` that emits an event.

- **Output:** an open `curl -N localhost:3000/events` terminal prints a line each time you create a note in another terminal.
- **Check:** kill and restart the server: the browser `EventSource` reconnects by itself.

Card 6.E: WebSocket chat room

- **Aim:** two-way real time.
- **Input:** Socket.IO server with `join(room)` and `message(room, text)` events and a token check in the handshake.
- **Output:** two browser tabs in the same room see each other's messages; a tab with a bad token is refused.
- **Check:** open three tabs, two in room A and one in room B; the third receives nothing from room A.

Self-check questions: Why can GraphQL be harder to cache than REST? When is SSE better than WebSocket? How does an idempotency key protect a payment retry?

Part 7: Caching, performance, and scaling

The rule of performance work: **measure first, change one thing, measure again.** Guessing wastes time.

7.1 Theory: caching

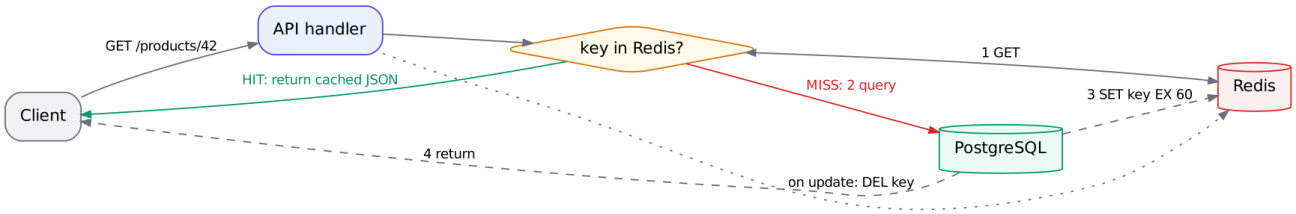
A cache stores the result of expensive work so repeated requests are cheap. The price is **staleness**: the cache may be out of date. Decide for each piece of data how stale is acceptable.

Layer	Example	Good for
Browser	`Cache-Control: max-age=3600`	Static assets, public data
CDN	Cloudflare, CloudFront, Fastly	Static files, cacheable public GET responses
Reverse proxy	Nginx, Varnish	Whole-response caching
In-process memory	`lru-cache`	Tiny hot data; lost on restart; not shared between instances

Layer	Example	Good for
Shared cache	Redis, Memcached	Query results, sessions, computed values shared by all instances
Database	Materialised views, query cache	Heavy reports

Strategies

- **Cache-aside (lazy loading)**: app checks the cache, on a miss reads the database and fills the cache. Most common (diagram below).
- **Write-through**: write to cache and database together; reads are always warm; writes are slower.
- **Write-behind**: write to cache, flush to the database later; fast but you can lose data.
- **Read-through**: the cache library itself loads missing values.



Hard problems

- **Invalidation.** Delete or update the key whenever the source changes, or accept a short TTL. Prefer short TTLs plus explicit deletes for important data.
- **Cache stampede.** A hot key expires and 1,000 requests hit the database at once. Fixes: single-flight (one request rebuilds, the rest wait), add random jitter to TTLs, or refresh before expiry.
- **Cache penetration.** Repeated requests for missing keys bypass the cache. Cache a short-lived "not found" marker.
- **Consistency.** A cached copy can disagree with the database for up to the TTL. Never cache data whose staleness causes harm (account balances in a payment decision).

7.2 Theory: making one instance fast

- **Keep handlers non-blocking** (Part 1). Offload CPU work to `worker_threads` with a pool (Piscina), or to a queue.

- **Database first.** Most slow endpoints are slow queries or N+1 patterns. Add indexes, select only needed columns, batch queries, use connection pooling.
- **Parallelise independent I/O** with `Promise.all`; add timeouts and limits so one slow dependency cannot hold every request.
- **Compress** responses (gzip or brotli) at the reverse proxy; **stream** large downloads instead of buffering.
- **Reduce payloads**: pagination, field selection, no giant nested arrays.
- **Keep-alive** connections to databases and upstream APIs (use an HTTP agent such as `undici`).
- **Profile** with the Node inspector, `clinic.js`, `0x` flamegraphs; look at event-loop delay (`perf_hooks.monitorEventLoopDelay`).

7.3 Theory: scaling out

Vertical scaling = a bigger machine. **Horizontal scaling** = more copies behind a load balancer. For horizontal scaling your app must be **stateless**: no user data in process memory, no files on local disk, no in-memory sessions. Put state in PostgreSQL, Redis, or object storage, and every instance becomes interchangeable.

- **Load balancer** distributes requests (round robin, least connections). Nginx, HAProxy, cloud load balancers.
- **Health checks**: `/healthz` (process alive) and `/readyz` (can reach DB and Redis). The balancer removes unhealthy instances.
- **Graceful shutdown**: on `SIGTERM` stop accepting new requests, finish in-flight ones, close DB pools, then exit.
- **Database scaling order**: indexes and query fixes, then caching, then read replicas, then partitioning or sharding. Sharding is the last resort.
- **Rate limiting algorithms**: fixed window (simple, bursty at edges), sliding window (smoother), **token bucket** (allows bursts, enforces average rate), leaky bucket (smooths output). Store counters in Redis so all instances share them.

7.4 Libraries and alternatives

Need	Pick	Alternatives
In-memory cache	`lru-cache`	`node-cache`, `quick-lru`, `@isaacs/ttlcache`

Need	Pick	Alternatives
Shared cache	Redis with `ioredis`	Memcached (`memjs`), Valkey (Redis fork), Dragonfly, KeyDB
Cache helpers	`cacheable`, `@nestjs/cache-manager`	Hand-written `getOrSet` helper
Single-flight / locks	`async-mutex` (in-process), Redis `SET NX PX` or `redlock` (distributed)	Postgres advisory locks
Rate limiting	`rate-limiter-flexible` (Redis-backed)	`express-rate-limit` + `rate-limit-redis`, `@fastify/rate-limit`
Worker threads	`piscina`	Raw `worker_threads`, `workerpool`
Multi-process	PM2, Node `cluster`	Containers per core on Kubernetes (preferred today)
Load testing	`autocannon`, k6	Artillery, wrk, Locust
Profiling	`clinic`, `0x`, Chrome DevTools via `--inspect`	Datadog or Pyroscope continuous profiling
HTTP client	`undici` (also behind global `fetch`)	`got`, `axios`, `ky`
CDN / edge	Cloudflare, CloudFront	Fastly, Bunny

7.5 Build cards

Card 7.A: cache-aside helper

- **Aim:** one reusable `getOrSet(key, ttlSeconds, loader)`.
- **Input:** a slow `loadProduct(id)` (simulate with a 300 ms delay or a heavy query).
- **Output:** first call 300 ms, later calls under 5 ms until the TTL ends; updating a product deletes its key.
- **Check:** log HIT or MISS; call 5 times and watch 1 MISS then 4 HITs; update then call again for a MISS.

Card 7.B: stampede protection

- **Aim:** survive a hot key expiring.
- **Input:** 200 parallel requests for the same expired key.
- **Output:** the loader runs once; all 200 receive the value.
- **Check:** a counter inside the loader prints 1.

Card 7.C: Redis rate limiter

- **Aim:** a limit shared by all instances.
- **Input:** limit 10 requests per minute per API key; run two server processes on different ports.
- **Output:** the 11th request within a minute gets `429` with `Retry-After`, whichever process receives it.
- **Check:** alternate requests between the two ports with a script.

Card 7.D: load test and graceful shutdown

- **Aim:** get a baseline and prove safe restarts.
- **Input:** `autocannon -c 100 -d 30` against `/notes`; handle `SIGTERM`.
- **Output:** a table of requests per second and p99 latency before and after caching; during a restart under load there are zero failed requests.
- **Check:** run autocannon, send `kill -TERM <pid>` mid-run, compare error counts.

Self-check questions: Which data would you never cache, and why? What makes an app stateless? Why is a token bucket friendlier to users than a fixed window?

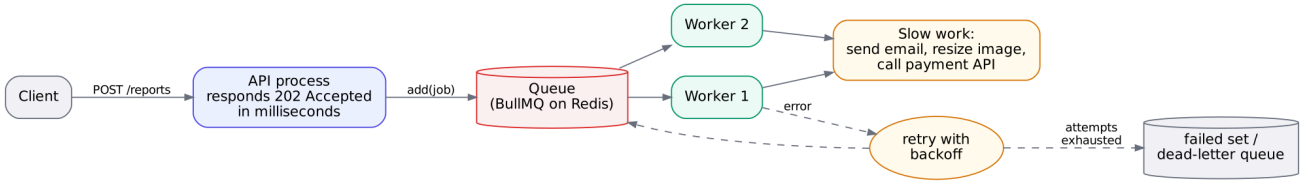
Part 8: Asynchronous processing (queues, jobs, events, outbox, sagas)

Not every task belongs inside the request. Sending an email, resizing an image, generating a report, or calling a slow partner API should happen **after** you answer the user. This part is the heart of scalable backend design.

8.1 Theory: why queues

- **Fast responses**: the API replies `202 Accepted` in milliseconds.
- **Reliability**: if the worker crashes, the job is not lost and is retried.
- **Smoothing spikes**: a burst of 10,000 requests becomes a backlog that workers drain at a steady rate.

- **Isolation**: heavy CPU work runs in separate worker processes and cannot freeze the API.
- **Decoupling**: the producer does not know who consumes the message.



Vocabulary

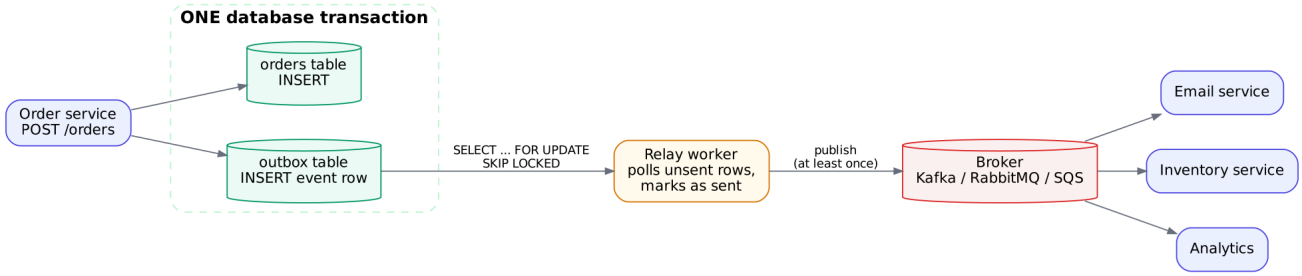
Term	Meaning
Producer / consumer (worker)	Adds a message / processes it
Work queue	Each message is processed by exactly one worker
Pub/Sub (topic)	Each subscriber group gets its own copy
Acknowledgement (ack)	Worker confirms success; without it the message returns to the queue
Retry with backoff	Try again after growing delays (1 s, 5 s, 30 s)
Dead-letter queue (DLQ)	Where messages go after the retries run out, for inspection
Delayed / scheduled job	Run at a future time or on a cron schedule
Concurrency	How many jobs one worker runs in parallel

Delivery guarantees. *At-most-once* can lose messages. *At-least-once* never loses but may deliver twice (the realistic default). *Exactly-once* is mostly a myth across systems; you get **effectively-once** by making consumers **idempotent**: processing the same message twice has the same effect as once (store processed message ids with a unique constraint, or write operations as "set" instead of "add").

8.2 Theory: events and the outbox pattern

In an **event-driven** design, a service announces facts (`OrderPlaced`) and others react. A classic bug: you save an order to the database, then publish an event, and the process crashes in between, so the event is lost. You cannot make a database and a broker commit

together. The **transactional outbox** solves it: write the business row and an `outbox` row in **one database transaction**; a relay process reads unsent outbox rows and publishes them, marking them sent. Delivery is at-least-once, so consumers must be idempotent.



8.3 Theory: sagas for multi-service workflows

A business action spanning services (reserve stock, charge card, create shipment) cannot use one database transaction. A **saga** is a sequence of local transactions, each with a **compensating action** that undoes it if a later step fails (refund the card, release the stock).

- **Choreography**: services react to each other's events. Simple for 2 or 3 steps; hard to follow as it grows.
- **Orchestration**: one coordinator tells each service what to do and handles failures. Easier to reason about; a workflow engine (Temporal) makes this durable.

8.4 Theory: choosing a message system

System	Model	Strengths	Weaknesses	Pick when
BullMQ (Redis)	Job queue	Delays, retries, priorities, repeatable jobs, dashboard (Bull Board), great Node API	Redis durability and memory limits	Background jobs in a Node app (the default choice)
pg-boss / Graphile Worker (PostgreSQL)	Job queue in your DB	No extra infrastructure, transactional enqueue with your data	Lower throughput than Redis	Moderate volume, want the outbox-like guarantee for free

System	Model	Strengths	Weaknesses	Pick when
RabbitMQ	Broker with exchanges and queues	Flexible routing, acks, mature	Operate a cluster; not a log	Complex routing, task distribution between services
Kafka (and Redpanda)	Distributed append-only log with partitions and consumer groups	Huge throughput, replay, ordering within a partition, event sourcing	Operational and conceptual weight	Event streaming, analytics pipelines, many consumers per event
AWS SQS + SNS, Google Pub/Sub	Managed queue / topics	No servers to run, scales automatically	Vendor lock-in; fewer features	You are on that cloud already
NATS (JetStream)	Lightweight messaging	Fast, simple	Smaller Node ecosystem	Low-latency service messaging
Temporal, Inngest, Trigger.dev	Durable workflow engines	Long-running, retried, observable workflows; sagas become code	Extra platform	Multi-step processes with waiting and human steps

8.5 Libraries

Need	Pick	Alternatives
Redis job queue	`bullmq` (+ `@nestjs/bullmq`)	`bee-queue`, `bull` (older)
Queue dashboard	Bull Board, Taskforce	Custom admin page
Postgres queue	`pg-boss`	`graphile-worker`
RabbitMQ	`amqplib`, `amqp-connection-manager`	`rascal`, `@nestjs/microservices` RMQ transport

Need	Pick	Alternatives
Kafka	`kafkajs`, `@confluentinc/kafka-javascript`	`@nestjs/microservices` Kafka transport
SQS	`@aws-sdk/client-sqs`	`sqs-consumer`
Cron	`croner`, `node-cron`	`@nestjs/schedule`, BullMQ repeatable jobs, system cron / Kubernetes CronJob
In-process events	`node:events`, `eventemitter3`	`@nestjs/event-emitter`
Workflows	`@temporalio/client` and worker SDK	Inngest, Trigger.dev, `workflow` libraries
Schema contracts for events	Zod / JSON Schema, Avro or Protobuf with a registry	CloudEvents format

8.6 Build cards

Card 8.A: email job with BullMQ

- **Aim:** move slow work out of the request.
- **Input:** `POST /users` creates a user then enqueues `send-welcome-email` (simulate sending with a 3 second delay).
- **Output:** the endpoint answers in under 50 ms; the worker (a separate process) logs "email sent" 3 seconds later.
- **Check:** stop the worker, create users, restart it: all queued emails are processed.

Card 8.B: retries and dead-letter

- **Aim:** failure handling.
- **Input:** a job that fails the first two times (throw on `attempt < 3`), `attempts: 5`, exponential backoff.
- **Output:** logs show attempts at growing intervals and success on the third; a job that always fails ends up in the failed set.
- **Check:** open Bull Board and see the failed job with its error.

Card 8.C: idempotent consumer

- **Aim:** survive duplicate delivery.
- **Input:** a `processed_messages(message_id PRIMARY KEY)` table; a handler that inserts the id and the business effect in one transaction.
- **Output:** delivering the same message 5 times changes the data once.
- **Check:** enqueue the same job id 5 times and compare row counts.

Card 8.D: transactional outbox

- **Aim:** never lose an event.
- **Input:** `POST /orders` writes `orders` and `outbox` in one transaction; a relay loop uses `SELECT ... FOR UPDATE SKIP LOCKED LIMIT 50` and publishes to a queue.
- **Output:** kill the app right after the transaction commits: after restart the relay still publishes the event.
- **Check:** add a `process.exit(1)` between commit and publish in a test, then restart.

Self-check questions: Why can a broker not give you true exactly-once delivery? What is the difference between a work queue and pub/sub? When would you choose Kafka over BullMQ?

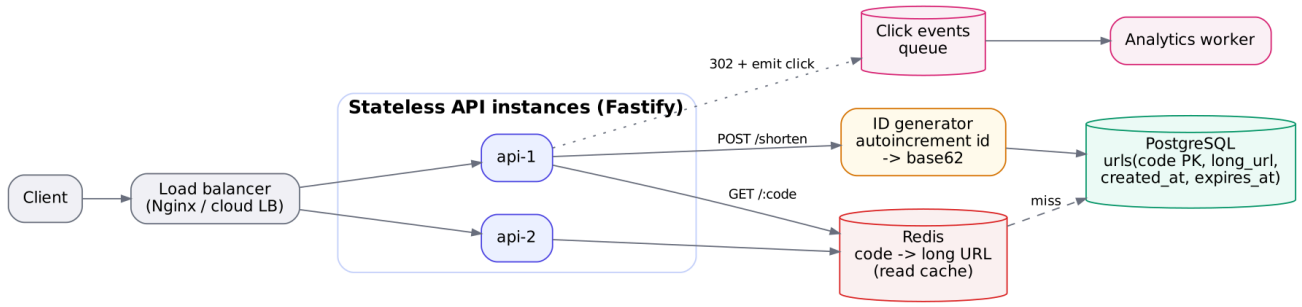
Part 9: System design scenarios (what to use, and the alternatives)

Interviews and real projects both ask the same thing: *given this requirement, which pieces do you pick and why?* Use this method every time:

1. **Clarify requirements**: functional (what it does) and non-functional (users, requests per second, latency, availability, data size, consistency needs).
2. **Estimate**: requests per second, storage per year, read/write ratio.
3. **Sketch the simplest design** that could work, then find the bottleneck.
4. **Name trade-offs** and alternatives out loud.

9.1 Scenario: URL shortener (read-heavy)

Requirements: create short links, redirect fast, optional expiry, click counts. Reads outnumber writes about 100 to 1.



- **ID generation:** auto-increment id encoded in base62 (`nanoid` or `base-x`), or random 7-character codes with a unique constraint and retry on collision. A counter service or Snowflake-style ids when you have several writers.
- **Storage:** PostgreSQL is plenty to billions of rows with an index on `code` . DynamoDB or Cassandra when you truly need massive scale.
- **Speed:** cache `code -> url` in Redis; return `301` (cacheable, fewer hits) or `302` (every click reaches you, better for analytics).
- **Analytics:** never write click counts in the redirect path. Emit an event to a queue (BullMQ or Kafka) and aggregate in a worker.
- **Abuse:** rate limit creation, validate URLs (no `javascript:`), block private IP targets.

Choice	Pick	Alternative
Framework	Fastify	Hono, Express
Database	PostgreSQL via Drizzle or Prisma	DynamoDB, MongoDB
Cache	Redis (`ioredis`)	In-process `lru-cache` per instance for the hottest codes
Analytics	BullMQ then a counters table	Kafka + ClickHouse

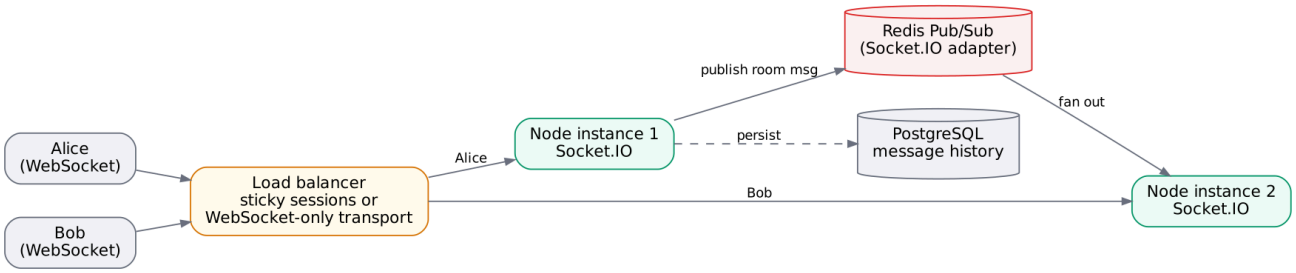
9.2 Scenario: file uploads and media

Never stream big files through your API servers into your own disk. Use **presigned URLs**: the API authorises the upload and returns a short-lived URL; the client uploads directly to object storage (S3, Cloudflare R2, Google Cloud Storage, MinIO); the API stores only metadata. Then a worker generates thumbnails.

- Libraries: `@aws-sdk/client-s3` + `@aws-sdk/s3-request-presigner`; `multer` or `@fastify/multipart` or `busboy` when you must accept multipart on the server; `sharp` for image resizing; `file-type` to check real content type; `tus` (resumable uploads) or S3 multipart for files over 100 MB; `clamav` for virus scanning.
- Checks: size limit, allowed MIME types, random object keys (never trust user filenames), private buckets with signed download URLs.

9.3 Scenario: real-time chat or notifications (stateful connections)

One server cannot hold every user, and Alice and Bob may be connected to different servers. Use **Redis pub/sub** (the Socket.IO Redis adapter) so a message published on instance 1 reaches the socket on instance 2. The load balancer needs sticky sessions for Socket.IO's HTTP long-polling fallback, or you force the WebSocket transport.



- **Persist** messages in PostgreSQL (or Cassandra/ScyllaDB at huge scale); push is a delivery optimisation, not the source of truth.
- **Presence** (online/offline) in Redis with TTL keys refreshed by heartbeats.
- **Offline users:** store notifications, send push via FCM/APNs, or email through a queue.
- **Alternatives:** managed services (Ably, Pusher, Supabase Realtime), Centrifugo, NATS, Redis Streams or Kafka when you need replay and ordering.

9.4 Scenario: search

- **Small to medium:** PostgreSQL full-text search (`tsvector` , GIN index) or trigram (`pg_trgm`) for typo tolerance.
- **Product-grade search** (typos, facets, ranking, autocomplete): Meilisearch or Typesense (simple, fast), Elasticsearch or OpenSearch (powerful, heavier), Algolia (hosted).
- **Sync strategy:** the database is the source of truth. Update the search index through events or an outbox worker, and design a re-index job from scratch for recovery.

- **Client libraries:** `meilisearch`, `typesense`, `@elastic/elasticsearch`, `@opensearch-project/opensearch`.

9.5 Scenario: payments and webhooks

- Use a provider (Stripe, Razorpay, Adyen) and **never touch raw card numbers**.
- Store your own `payments` table with a status machine (`pending`, `succeeded`, `failed`, `refunded`); only forward transitions.
- **Webhooks:** verify the signature using the raw request body (so keep the raw bytes, not the parsed JSON), respond `200` quickly, push the work to a queue, and handle duplicates by event id.
- **Idempotency keys** on every create call (Part 6). Reconcile with the provider daily because webhooks can be missed.
- Money: store integers in the smallest unit (paise or cents) plus currency; never floating point.

9.6 Scenario: scheduled and long-running jobs

- Cron-like tasks: BullMQ repeatable jobs or a Kubernetes CronJob. Ensure only **one** instance runs the task (a distributed lock or a queue guarantees it).
- Long workflows with waits and retries (onboarding emails over 14 days, order fulfilment): Temporal or Inngest instead of hand-written timers.
- Reports and exports: enqueue a job, write the file to object storage, notify the user with a download link.

9.7 Scenario: multi-tenant SaaS

Model	How	Pros	Cons
Shared tables with `tenant_id`	One schema; every query filters by tenant	Cheapest, simplest	One missed filter leaks data; use PostgreSQL **row-level security** as a safety net
Schema per tenant	Separate PostgreSQL schema	Better isolation	Migrations multiply

Model	How	Pros	Cons
Database per tenant	Separate DB	Strongest isolation, per-tenant backup	Costly, connection management

Resolve the tenant once (subdomain, header, or token claim), carry it in `AsyncLocalStorage`, and add tests that try to read another tenant's data.

9.8 Scenario: high-traffic read API (feed, catalogue)

CDN for public GETs, Redis for hot objects, read replicas for the database, pagination with cursors, precomputed feeds written by background workers ("fan-out on write") for users with few followers and computed at read time ("fan-out on read") for celebrities. Measure with real traffic shapes before choosing.

9.9 Quick pattern-to-library map

Technique	Typical Node library	Other options
Retry with backoff	`p-retry`, `async-retry`	BullMQ built-in for jobs
Circuit breaker (stop calling a failing service)	`opossum`	`cockatiel` (also retry, timeout, bulkhead)
Timeout / cancellation	`AbortSignal.timeout(ms)`	`p-timeout`
Concurrency limit	`p-limit`, `p-queue`	`bottleneck`
Distributed lock	`redlock`	Postgres advisory locks
Feature flags	OpenFeature, Unleash, GrowthBook	Config table
Request context	`AsyncLocalStorage`	`nestjs-cls`
Dependency injection outside Nest	`awilix`, `tsyringe`, `inversify`	Manual wiring (often enough)
Domain events	`eventemitter3`	`@nestjs/event-emitter`
Date and money maths	`date-fns`, `dinero.js` or integers	`big.js`, `decimal.js`

Technique	Typical Node library	Other options
Email	`nodemailer` + provider (SES, Resend, Postmark, SendGrid)	Templating with `mjml`, `react-email`
PDF / reports	`pdfkit`, Puppeteer or Playwright render	`pdf-lib`
Web scraping / HTTP	`undici`, `cheerio`	Playwright
Geospatial	PostGIS	`h3-js`, Redis GEO

9.10 Build cards (design practice)

Card 9.A: build the URL shortener end to end

- **Aim:** apply Parts 2 to 8 in one project.
- **Input:** `POST /shorten {url}`, `GET /:code`, `GET /stats/:code`.
- **Output:** redirects served from Redis after the first hit; click events flow through a queue; creation limited to 10 per minute per IP.
- **Check:** load test the redirect with autocannon; record requests per second with and without Redis.

Card 9.B: presigned upload

- **Aim:** keep file bytes off your server.
- **Input:** `POST /uploads {filename, contentType, size}` and a local MinIO container (S3 compatible).
- **Output:** the API returns a presigned `PUT` URL; `curl --upload-file` to that URL succeeds; a worker creates a 200 px thumbnail with `sharp`.
- **Check:** reject a 50 MB file and a `.exe` content type at the API step.

Card 9.C: chat across two instances

- **Aim:** prove cross-server broadcast.
- **Input:** run two Socket.IO servers on ports 3001 and 3002 with the Redis adapter.
- **Output:** a client on 3001 and a client on 3002 in the same room receive each other's messages.
- **Check:** stop Redis and observe what breaks; restart and see recovery.

Card 9.D: Stripe-style webhook receiver

- **Aim:** secure and idempotent webhooks.
- **Input:** a `POST /webhooks/pay` route that reads the raw body and an `X-Signature` HMAC header.
- **Output:** bad signature gives `400`; duplicate event id is ignored; valid event enqueues a job and returns `200` in under 100 ms.
- **Check:** replay the same payload twice with a script and compare database rows.

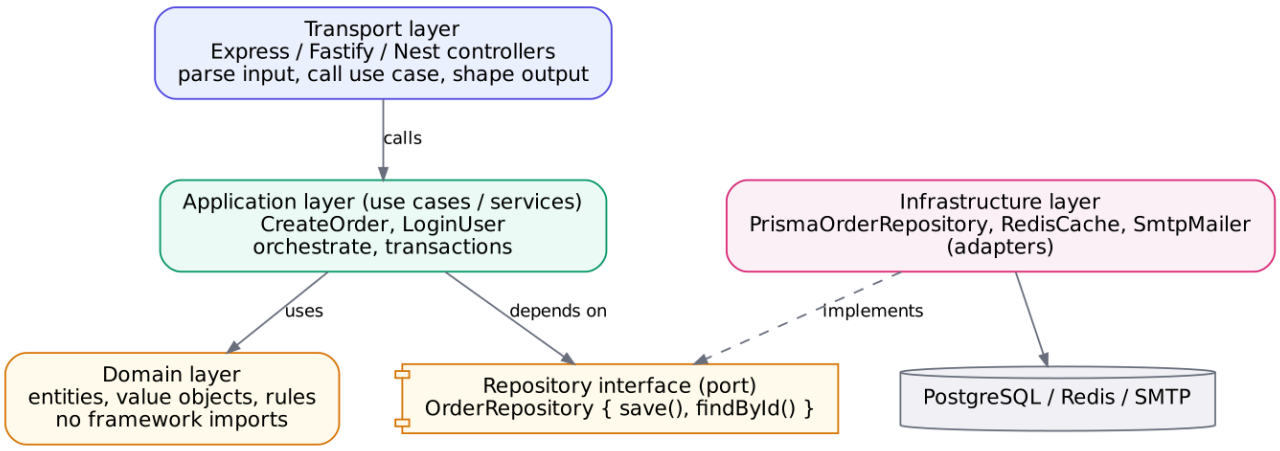
Self-check questions: Why presigned URLs instead of uploading through the API? Why is the database, not the search engine, the source of truth? What can go wrong with shared-table multi-tenancy, and what is the safety net?

Part 10: Architecture patterns

Architecture is how you split code so that change stays cheap. The goal is not purity; it is to keep business rules independent of frameworks and databases so each can change without rewriting the rest.

10.1 Layered and clean (hexagonal) architecture

The dependency rule: **inner layers never import outer layers.** Business rules do not know about Express, Prisma, or Redis. They know interfaces (**ports**); the outer layer supplies **adapters** that implement them.



A folder layout that works for both Fastify and Nest, organised **by feature** (not by technical type):

```
src/
  modules/
    orders/
      orders.controller.ts     # transport: HTTP in/out
      orders.service.ts        # use cases, transactions
      orders.repository.ts     # interface + Prisma implementation
      orders.schemas.ts        # Zod schemas / DTOs
      orders.events.ts         # domain events
      orders.test.ts
    users/ ...
  shared/ (errors, config, logger, db, queue)
  main.ts (compose everything)
```

Benefits: you can test services with a fake repository (no database), swap Prisma for Drizzle by writing a new adapter, and understand one feature by opening one folder.

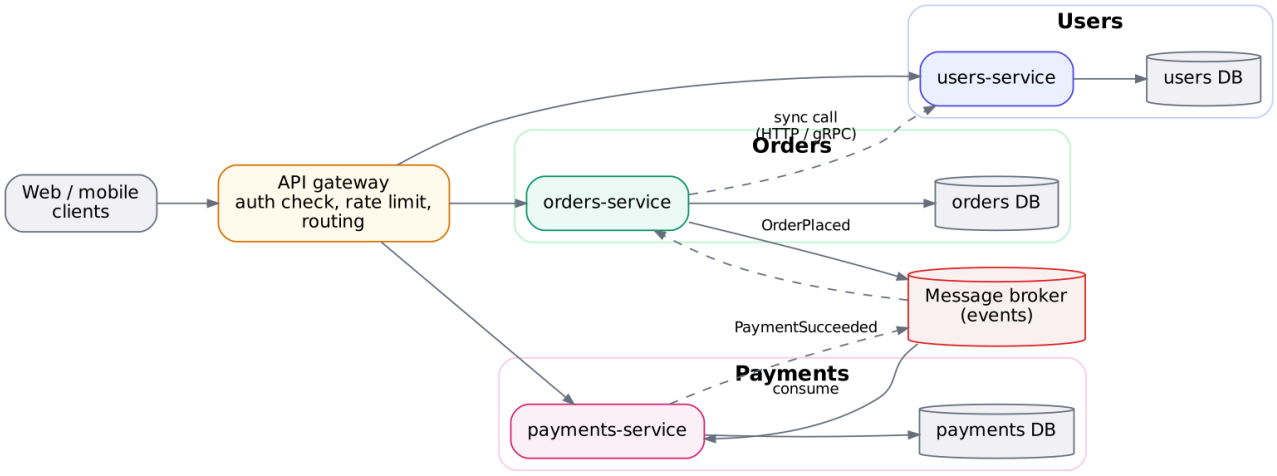
Domain-Driven Design, the useful parts. Talk to domain experts and use their words (the *ubiquitous language*). Group code into **bounded contexts** (Billing and Shipping may both have a "Customer" that means different things). Inside, model **entities** (have identity), **value objects** (compared by value, like `Money`), and **aggregates** (a cluster changed as one unit through a root; one transaction per aggregate). You do not need the whole DDD vocabulary for CRUD apps; borrow ideas as complexity grows.

10.2 Monolith, modular monolith, microservices

	Monolith	Modular monolith	Microservices
Deployment	One unit	One unit, strict module boundaries	Many independent units
Data	One database	One database, tables owned per module	Database per service
Calls between parts	Function calls	Calls through module public APIs or in-process events	Network (HTTP, gRPC, events)
Team scaling	Hard past a point	Good	Best for many teams

	Monolith	Modular monolith	Microservices
Operational cost	Lowest	Low	High: tracing, deploys, versioning, network failures, data consistency
Start here?	For tiny apps	**Yes, for most products**	Only when a real need appears (team size, independent scaling, fault isolation)

Honest advice: begin with a modular monolith. Extract a service only when a boundary is stable and you have a concrete reason. Microservices trade code complexity for operational and consistency complexity; many teams regret splitting too early. NestJS modules map cleanly onto this path.



What microservices require

- **API gateway / BFF** for auth, rate limiting, routing, and aggregation (Kong, Traefik, Nginx, AWS API Gateway, or a small Fastify/Express service).
- **Service-to-service calls** with timeouts, retries, circuit breakers (`opossum`, `cockatiel`); prefer async events when the caller does not need an answer now.
- **Data ownership**: no shared tables; use events and sagas (Part 8) for cross-service consistency.
- **Contracts**: OpenAPI, Protobuf, or event schemas with versioning; consumer-driven contract tests (Pact).
- **Observability**: distributed tracing with a correlation id (Part 11).

- **Service discovery and config**: Kubernetes DNS and ConfigMaps, or Consul.

10.3 CQRS and event sourcing (advanced, use sparingly)

CQRS separates the model used to change data (commands) from the model used to read it (queries), often with different stores: write to PostgreSQL, read from a denormalised view or search index kept up to date by events. It helps when reads and writes have very different shapes or loads. **Event sourcing** stores the *sequence of events* as the truth and rebuilds state by replaying them; it gives audit and time travel but makes queries, migrations, and mistakes harder. Both add real complexity; adopt them for a specific pain, not for style. NestJS has `@nestjs/cqrs`; EventStoreDB and Kafka are common event stores.

10.4 Other patterns worth knowing

Pattern	Idea	Use
Repository	Hide persistence behind an interface	Testability, swapping databases
Unit of work	Group changes into one transaction	Consistent aggregates
Dependency injection	Receive dependencies instead of creating them	Tests, flexibility (Nest, awilix, or manual wiring)
Strangler fig	Route traffic piece by piece from an old system to a new one	Migrations without a rewrite
Bulkhead and circuit breaker	Isolate failures; stop calling failing dependencies	Resilience
Backend for Frontend	One API tailored per client type	Web versus mobile needs
Feature flags	Deploy code dark, release by config	Safe rollouts
Twelve-factor app	Config in env, stateless processes, logs to stdout, disposable processes	Cloud-ready services

10.5 Build cards

Card 10.A: refactor Notes into layers

- **Aim:** feel the dependency rule.
- **Input:** the Part 4 Notes API with SQL inside route handlers.
- **Output:** `NotesController` (HTTP only) calls `NotesService` (rules) which depends on a `NotesRepository` interface; `PrismaNotesRepository` implements it.
- **Check:** write a unit test for `NotesService` using an in-memory fake repository; it runs in milliseconds with no database.

Card 10.B: two modules, one boundary

- **Aim:** a modular monolith.
- **Input:** `orders` and `inventory` modules; placing an order must reserve stock.
- **Output:** `orders` calls only `inventory`'s exported interface (`reserve(sku, qty)`), never its tables; a lint rule (`eslint-plugin-boundaries` or `dependency-cruiser`) forbids deep imports.
- **Check:** try a forbidden import and confirm the linter fails the build.

Card 10.C: first extraction

- **Aim:** experience a service boundary.
- **Input:** move `inventory` to its own process with its own database, exposed over HTTP or gRPC.
- **Output:** `orders` calls it with a 1 second timeout, one retry, and a circuit breaker; stop the inventory service and `orders` returns a clean `503`, not a hang.
- **Check:** kill the inventory container during a load test and read the error rate.

Self-check questions: What does the dependency rule forbid? Give two reasons to stay a modular monolith. What problem does a saga solve that a database transaction cannot?

Part 11: Testing, observability, and deployment

Code is finished only when it is tested, visible in production, and shippable by a pipeline.

11.1 Theory and tools: testing

The test pyramid: many fast **unit tests** (pure logic, services with fake repositories), a moderate number of **integration tests** (HTTP routes with a real database), and few **end-to-end tests** (the whole system). Also add **contract tests** between services and **load tests** before launches.

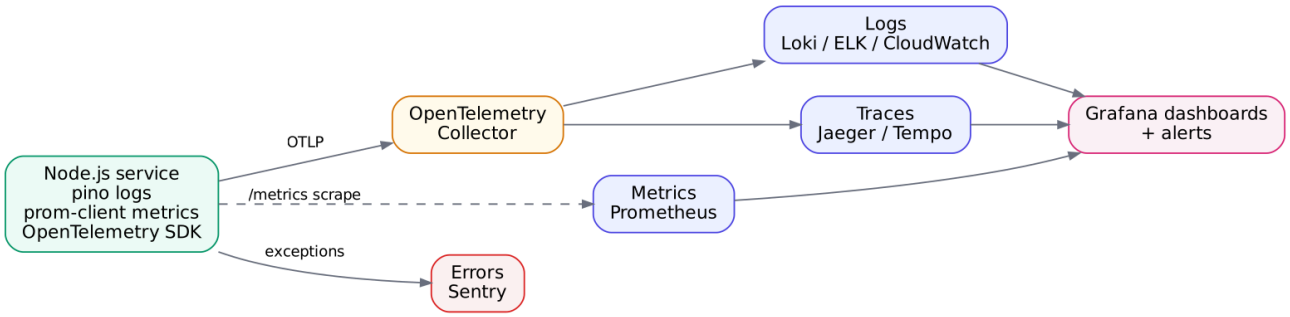
Need	Pick	Alternatives
Test runner	Vitest	Jest, `node:test` (built in), Mocha
HTTP tests (Express)	`supertest`	`light-my-request` (Fastify `app.inject`), `@nestjs/testing` + supertest
Real dependencies	Testcontainers (PostgreSQL, Redis in Docker)	docker-compose in CI, `pg-mem` (fake, less faithful)
Mocking outbound HTTP	`msw`, `nock`, undici `MockAgent`	Hand-written fakes
Test data	`@faker-js/faker`, factory functions	Fishery
Contract tests	Pact	OpenAPI-based validators
Load tests	k6, autocannon	Artillery
Coverage	`v8` coverage (built in to Vitest/Jest)	c8, nyc

Good habits: each test creates its own data and cleans up (transaction rollback or truncate); test behaviour, not implementation; do not mock the database in integration tests (it hides real SQL bugs); keep the suite under a few minutes.

11.2 Theory and tools: observability

When production misbehaves you need three signals:

- **Logs**: what happened (structured JSON, with request id).
- **Metrics**: numbers over time. Track the **RED** set per endpoint: **R**ate, **E**rrors, **D**uration (p50, p95, p99). Also CPU, memory, event-loop delay, DB pool usage, queue depth.
- **Traces**: the path of one request across services, with timing per step.



Need	Pick	Alternatives
Logger	`pino`	`winston`
Metrics	`prom-client` + Prometheus + Grafana	Datadog, New Relic, OpenTelemetry metrics
Tracing and instrumentation	OpenTelemetry (`@opentelemetry/sdk-node` with auto-instrumentations)	Datadog APM, Elastic APM
Trace backend	Jaeger, Grafana Tempo	Honeycomb, Datadog
Error tracking	Sentry	Rollbar, Bugsnag
Log storage	Grafana Loki, Elasticsearch/OpenSearch (ELK)	CloudWatch, Datadog Logs
Uptime / alerts	Grafana alerts, Alertmanager, PagerDuty, UptimeRobot	Cloud provider alarms

Alert on **symptoms users feel** (error rate, latency, queue backlog age), not on every CPU blip. Define a service level objective (for example 99.9 percent of requests succeed under 500 ms) and alert when you burn its budget too fast.

11.3 Theory and tools: deployment

- **Containers.** Package the app in a Docker image so it runs identically everywhere. Use a multi-stage build, a slim base image, a non-root user, and copy only what you need.
- **Config and secrets** come from environment variables injected at run time (never baked into the image).
- **Health endpoints:** `/healthz` and `/readyz`, plus graceful shutdown on `SIGTERM` (Part 7).

- **CI/CD:** on every pull request run lint, type check, tests, build the image; on merge deploy automatically, with a way to roll back.
- **Database migrations** run as a separate release step before the new code starts, and must be backward compatible with the old code (Part 4).
- **Environments:** local, staging (production-like), production.

```
# syntax=docker/dockerfile:1
FROM node:22-slim AS build
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN corepack enable && pnpm install --frozen-lockfile
COPY . .
RUN pnpm build && pnpm prune --prod

FROM node:22-slim
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
USER node
EXPOSE 3000
CMD ["node", "dist/main.js"]
```

Where to run	Good for	Notes
Railway, Render, Fly.io	Fast start, small teams	Managed deploys, simple scaling
AWS ECS/Fargate, Google Cloud Run, Azure Container Apps	Serious production without managing Kubernetes	Container in, autoscaling out
Kubernetes (EKS, GKE, AKS)	Many services, platform teams	Powerful, high operational cost
Serverless functions (AWS Lambda, Vercel, Cloudflare Workers)	Spiky or low traffic, event handlers	Cold starts, connection pooling and time limits need care
VPS with Docker Compose and Nginx	Cheap and understandable	You handle patching and backups

Other tools: GitHub Actions or GitLab CI for pipelines; Terraform or Pulumi for infrastructure as code; Nginx or Caddy as reverse proxy with automatic TLS; PM2 if you run without containers.

11.4 Build cards

Card 11.A: integration tests with a real database

- **Aim:** trustworthy tests.
- **Input:** the Notes API, Vitest, supertest, Testcontainers PostgreSQL.
- **Output:** tests for create, validation failure, not found, and pagination; each starts from a clean database.
- **Check:** `pnpm test` passes from a fresh clone with only Docker installed; break an SQL query on purpose and see a test fail.

Card 11.B: metrics and a dashboard

- **Aim:** see your service.
- **Input:** `prom-client` default metrics plus an `http_request_duration_seconds` histogram labelled by route and status; a `docker-compose` with Prometheus and Grafana.
- **Output:** a Grafana panel showing requests per second and p95 latency per route.
- **Check:** run autocannon and watch the graph move; slow one route on purpose and find it on the dashboard.

Card 11.C: tracing one request

- **Aim:** follow a request through API, queue, and worker.
- **Input:** OpenTelemetry auto-instrumentation and Jaeger in Docker.
- **Output:** one trace shows the HTTP span, the SQL span, and the job span with timings.
- **Check:** find the slowest span for a deliberately slow query.

Card 11.D: containerise and ship

- **Aim:** a pipeline from commit to running container.
- **Input:** the Dockerfile above and a GitHub Actions workflow with jobs `lint`, `test`, `build`, `deploy`.
- **Output:** a push to `main` builds an image, runs migrations, and deploys; the image runs as a non-root user and is under about 250 MB.

- **Check:** `docker run` with a missing env variable must fail fast with a clear message (Card 3.C).

Self-check questions: Why test against a real database? What do RED metrics tell you that logs alone do not? Why must migrations be backward compatible with the previous release?

Part 12: Capstone project, study plan, and cheat sheet

12.1 Capstone: a hiring platform API

One project that touches almost every part: companies post jobs, candidates apply with a resume, resumes are parsed in the background, and recruiters search and rank candidates.

Feature	Parts it exercises	Suggested stack
Register, login, roles (candidate, recruiter, admin), refresh rotation	5	Fastify or Nest, argon2, jose, Redis sessions or JWT
Jobs and applications CRUD with validation and cursor pagination	2, 3, 4, 6	PostgreSQL, Drizzle or Prisma, Zod, OpenAPI
Resume upload with presigned URLs and size and type checks	9	S3-compatible storage (MinIO locally)
Background resume parsing and text extraction, retries, dead-letter	8	BullMQ workers, idempotent handlers
Search jobs and candidates, filters and ranking	9	PostgreSQL full-text first, then Meilisearch or Typesense
Multi-tenant companies with row-level security	9	Shared tables with `tenant_id` and PostgreSQL RLS
Email notifications and status changes through the outbox	8	Outbox table, relay, queue, nodemailer
Rate limits, caching of hot job listings	7	Redis, `rate-limiter-flexible`

Feature	Parts it exercises	Suggested stack
Tests, metrics, tracing, Docker, CI	11	Vitest, Testcontainers, OpenTelemetry, GitHub Actions
Real-time status updates for recruiters	6	SSE first, Socket.IO if you need two-way

Milestones

1. Skeleton, config, logging, health checks, Docker Compose with PostgreSQL and Redis.
2. Auth and roles, then jobs and applications.
3. Uploads and the parsing worker.
4. Search, caching, rate limiting.
5. Outbox and notifications.
6. Tests, dashboards, CI, deploy. Write a one-page design document with the trade-offs you chose and the alternatives you rejected.

12.2 A 12-week plan

Week	Part	Deliverable
1	1	Cards 1.A to 1.C
2	2	The Notes API in all three frameworks
3	3	Validation, errors, config, logging added
4	4	PostgreSQL, relations, transaction, index measurements
5	5	Full auth with refresh rotation and hardening
6	6	OpenAPI, idempotent POST, SSE, one GraphQL or tRPC variant
7	7	Cache helper, stampede test, Redis rate limiter, load test
8	8	BullMQ jobs, idempotent consumer, outbox
9	9	URL shortener and presigned uploads

Week	Part	Deliverable
10	10	Layered refactor and module boundaries
11	11	Tests, metrics, tracing, Docker, CI
12	Capstone	Finish, deploy, write the design document

12.3 Decision cheat sheet

If you need...	Reach for...	Consider instead...
A quick, understandable API	Express 5	Hono
High throughput and schema-first design	Fastify	Hono on Node
Large team structure and DI	NestJS (on the Fastify adapter if speed matters)	Fastify with feature folders
Relational data	PostgreSQL	MySQL
Type-safe database access	Prisma or Drizzle	Kysely, raw `pg`
Flexible document data	MongoDB with Mongoose	PostgreSQL `jsonb`
Sessions, cache, counters, pub/sub	Redis	Valkey, Memcached, Dragonfly
Background jobs in Node	BullMQ	pg-boss, Temporal, Inngest
High-volume event streaming	Kafka	Redpanda, NATS JetStream, SQS/SNS
Login for a product	Hosted provider or Better Auth	Passport + your own sessions
Typed validation	Zod	Valibot, TypeBox, class-validator
Real-time two-way	Socket.IO + Redis adapter	`ws`, Ably, Centrifugo
One-way push	SSE	WebSocket

If you need…	Reach for…	Consider instead…
Service-to-service RPC	gRPC or Connect	HTTP + OpenAPI, tRPC (TypeScript only)
Search	PostgreSQL full-text, then Meilisearch	Elasticsearch, Typesense
File storage	S3-compatible object storage	Local disk (single server only)
Logging	pino	winston
Observability	OpenTelemetry + Grafana stack	Datadog, New Relic
Tests	Vitest + supertest + Testcontainers	Jest, `node:test`
Deployment	Containers on Cloud Run, ECS, Fly.io, or Render	Kubernetes, serverless

12.4 Common mistakes checklist

- Business logic inside controllers; SQL inside handlers.
- Trusting client input, or skipping ownership checks.
- `async` handlers without error handling (Express 4), unhandled promise rejections.
- Blocking the event loop with sync or CPU-heavy code.
- One connection per request instead of a pool; N+1 queries; missing indexes on foreign keys.
- Storing state in process memory or on local disk, then scaling horizontally.
- Retrying non-idempotent operations; publishing events outside the transaction that creates them.
- Caching without an invalidation plan or TTL.
- Secrets in the repository; wide-open CORS; no rate limits on login.
- Logging sensitive data; no request ids; no metrics until the first outage.
- Microservices before a modular monolith; a shared database between services.
- Migrations that break the running version.

12.5 Where to keep learning

The official documentation is the most reliable source and stays current: the Node.js docs and guides, the Express, Fastify, and NestJS documentation, the PostgreSQL manual (especially the chapters on indexes and concurrency control), the Redis documentation, and the OWASP Top 10 and Cheat Sheet Series for security.

Library names and capabilities in this report come from general knowledge, not from pages I opened while writing, so confirm current versions and option names in each project's documentation before you install. Good books to read alongside: *Designing Data-Intensive Applications* by Martin Kleppmann (the best single book on the ideas behind Parts 4, 7, 8, and 9) and *Node.js Design Patterns* by Mario Casciaro and Luciano Mammino.